

TALLER EVALUATIVO: PIPELINES DE CI/CD & DESPLIEGUE CONTENERIZADO (TIEMPO: 2 HORAS)

Ingeniería de Software V

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS
INTELIGENTES

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS

INFORMACIÓN DE LA ACTIVIDAD

IDENTIFICACIÓN

CÓDIGO:	09776 - TIC	PERÍODO:	202620 (Semestre 2026-2)
NRC / GRUPO:	11005 / Grupo 001	PONDERACIÓN:	10.0 % (Taller Evaluativo) • Duración: 2 Horas
POLÍTICA IAG:	Nivel 3: Colaboración Asistida	CALIFICACIÓN:	_____ / 5.0
ESTUDIANTE 1:	_____	CÓDIGO:	_____
ESTUDIANTE 2:	_____	CÓDIGO:	_____

1 Contexto del Problema & Arquitectura Multi-Tier

1.1 Descripción del Desafío de Ingeniería DevOps

La organización **Icesi Tech Solutions** requiere modernizar su plataforma de inventario de productos compuesta por dos componentes desacoplados:

- Backend REST** (`product-api`): Microservicio en **Spring Boot 3 (Java 17)** con JPA y base de datos H2 en memoria, encargado de exponer operaciones CRUD de productos (GET, POST, PUT, DELETE `/api/products`).
- Frontend SPA** (`product-frontend`): Interfaz de usuario en **React 18 + Vite + TypeScript** que consume el API a través de la variable de entorno `VITE_API_URL`.

Actualmente el equipo no posee automatización: las pruebas, compilaciones y despliegues se ejecutan manualmente. El objetivo de este taller es estructurar el ciclo completo de entrega continua implementando **Multi-Stage Dockerfiles**, orquestación con **Docker Compose** y un pipeline declarativo de 4 jobs en **GitHub Actions**.

Componente	Tecnología & Stack	Puerto / Endpoint
Backend REST	Java 17, Spring Boot 3.2, Spring Data JPA, H2	Puerto 8080 (<code>/api/products</code>)
Frontend SPA	React 18, TypeScript, Vite 5, Nginx Alpine	Puerto 3000 (Proxy: /)
Orquestación	Docker Compose v2 (Red Bridge <code>product-network</code>)	<code>docker compose up -d</code>
CI/CD Pipeline	GitHub Actions (<code>.github/workflows/ci-cd.yml</code>)	4 Jobs (Test, Lint, Build, Smoke)

2 Procedimiento Experimental & Tareas a Desarrollar

2.1 Fase 1: Pruebas Unitarias y Compilación de Artefactos (30 min)

En el código base suministrado en `codigo.base/`:

- a) En `backend/`, ejecute la suite de pruebas unitarias y de controlador con Maven:

```
cd backend && mvn clean test
```

Verifique que `ProductServiceTest` y `ProductControllerTest` reporten 100 % de éxito.

- b) En `frontend/`, valide la consistencia de tipos TypeScript y genere el build de producción:

```
cd frontend && npm install && npm run lint && npm run build
```

2.2 Fase 2: Contenerización Multi-Stage con Docker (30 min)

Diseñe y optimice los archivos `Dockerfile` aplicando buenas prácticas de reducción de peso y seguridad:

■ **Backend** (`backend/Dockerfile`):

- *Stage 1 (Builder)*: `maven:3.9.6-alpine`. Descarga de dependencias en caché y compilación (`mvn package -DskipTests`).
- *Stage 2 (Runtime)*: `eclipse-temurin:17-jre-alpine`. Copia del archivo `.jar`, usuario no root (`appuser`) y puerto 8080.

■ **Frontend** (`frontend/Dockerfile`):

- *Stage 1 (Builder)*: `node:20-alpine`. Instalación limpia (`npm install`) y generación de estáticos (`npm run build`).
- *Stage 2 (Runtime)*: `nginx:alpine`. Copia de estáticos `dist/` hacia Nginx con soporte de fallback SPA en `nginx.conf`.

2.3 Fase 3: Orquestación Multicontenedor con Docker Compose (20 min)

Complete el archivo `docker-compose.yml` para orquestar la ejecución conjunta de los servicios:

- a) Declare una red compartida tipo puente (`product-network`).
- b) Configure el *healthcheck* en el backend validando la respuesta HTTP en `http://localhost:8080/api/products`.
- c) Configure la directiva `depends_on` con `condition: service_healthy` en el contenedor frontend.
- d) Levante el entorno completo y compruebe su funcionamiento en el navegador (`http://localhost:3000`):

```
docker compose up -d --build
docker compose ps
curl -i http://localhost:8080/api/products
```

2.4 Fase 4: Pipeline Declarativo en GitHub Actions (30 min)

Configure el flujo de automatización en `.github/workflows/ci-cd.yml` estructurado en 4 etapas:

- a) **Job 1 (test-backend):** Setup JDK 17 (Temurin), ejecución de `mvn test` y carga del artefacto JAR (`upload-artifact`).
- b) **Job 2 (test-frontend):** Setup Node 20, verificación estática (`npm run lint`) y compilación del bundle.
- c) **Job 3 (build-docker):** Construcción de imágenes Docker utilizando `docker/build-push-action` con tags versionados.
- d) **Job 4 (smoke-test):** Despliegue con `docker compose up -d` y validación de disponibilidad mediante consultas `curl` automatizadas.

3 Cuestionario de Análisis & Conceptos DevOps (10 min)

Responda brevemente a las siguientes preguntas con base en la experiencia de laboratorio:

- 1. **Inmutabilidad y Multi-Stage:** ¿Por qué no se debe incluir el SDK completo de Java (JDK) ni el compilador Maven en la imagen Docker final de producción? ¿Qué impacto tiene esta decisión en el tamaño de la imagen y en la superficie de ataque (CVEs)?
- 2. **Shift-Left Testing en CI:** Explique qué sucede si un desarrollador introduce un cambio que rompe una prueba unitaria en el Backend. ¿Cómo protege el pipeline de GitHub Actions la estabilidad de la rama `main`?
- 3. **Pruebas de Humo (Smoke Testing):** ¿Cuál es la diferencia fundamental entre una prueba unitaria (*Unit Test*) ejecutada en la fase de build y una prueba de humo (*Smoke Test*) ejecutada tras levantar los contenedores?

4 Rúbrica de Evaluación Modularizada por Punto (Escala 0.0 a 5.0)

Punto Entregable	Pts	Excelente (4.5 – 5.0)	Aceptable (3.0 – 4.4)	Insuficiente (0.0 – 2.9)
2.1. Dockerfile Backend	0.5	Multi-stage impecable (Maven builder → JRE 17 slim), usuario no root (appuser) y cache.	Multi-stage funcional pero usa imagen pesada (JDK) o ejecuta como root.	Dockerfile de una sola etapa o genera imágenes de tamaño excesivo (>500MB).
2.2. Dockerfile Frontend	0.5	Multi-stage eficiente (Node → Nginx Alpine), archivo nginx.conf con fallback SPA.	Frontend en Nginx funcional pero sin configuración adecuada de SPA.	Servidor web no levanta o sirve estáticos de forma incorrecta (404 en rutas).
3.1. Docker Compose	1.2	Orquestación completa (product-network), healthchecks condicionales y puertos funcionales.	Stack levanta pero sin healthchecks o dependencias declaradas.	Error al levantar el stack o conflicto de puertos y redes en Docker.
4.1. GitHub Actions CI	1.8	Workflow declarativo con 4 jobs (Test, Lint, Docker Build, Smoke Test) en verde (Passing).	Pipeline funcional pero con jobs secuenciales innecesarios o sin cache.	Pipeline falla, no ejecuta pruebas de humo o sintaxis YAML incorrecta.
5.1. Análisis Técnico	1.0	Respuestas técnicas rigurosas sobre inmutabilidad y Shift-Left, con bitácora ética de IAG.	Respuestas breves o bitácora de prompts incompleta.	Respuestas erróneas conceptualmente o no incluye análisis.
CALIFICACIÓN TOTAL	5.0	Escala Institucional Universidad Icesi: Aprobatoria ≥ 3.0 • Duración Máxima: 2 Horas		

INSTRUCCIONES DE ENTREGA

MODALIDAD

- Repositorio en GitHub:** Subir el código base con las soluciones implementadas y verificar que el workflow de **GitHub Actions** esté en Verde (**Passing**).
- Documento de Resultados (PDF):** Adjuntar en la plataforma de entrega este documento diligenciado con:
 - Enlace al repositorio privado en la organización con acceso al docente.
 - Captura de pantalla de la ejecución exitosa de los 4 jobs de GitHub Actions.
 - Respuestas al cuestionario de análisis técnico de la Sección 3.